

ORM, Hibernate et JPA

ORM

- ORM - Object Relational Mapping
- JPA - Java Persistence API
- Avantages et inconvénients de Hibernate
- Objet - Relationnel - 'mismatch'

ORM C'est quoi ?

- Mécanisme qui permet de sauvegarder ou plutôt persister des objets dans une base de donnée relationnelle
- On sauve la représentation d'objets

JPA et Hibernate

- JPA - Spécification java pour persister des objets
- Hibernate - Implémentation JPA la plus populaire. Il existe d'autres implémentations de la norme JPA autre qu'Hibernate
- Un mécanisme de 'mapping' est utilisé pour stocker les objets dans une base de donnée relationnelle.

Avantages de JPA

- On écrit moins de code
- Le développement est plus rapide
- Moins besoin de jouer avec du SQL
- Permet de se consacrer à la logique d'affaire plutôt qu'aux détails de base de données.
- Indépendant de la base de donnée (Oracle vs MySql vs Postgres)

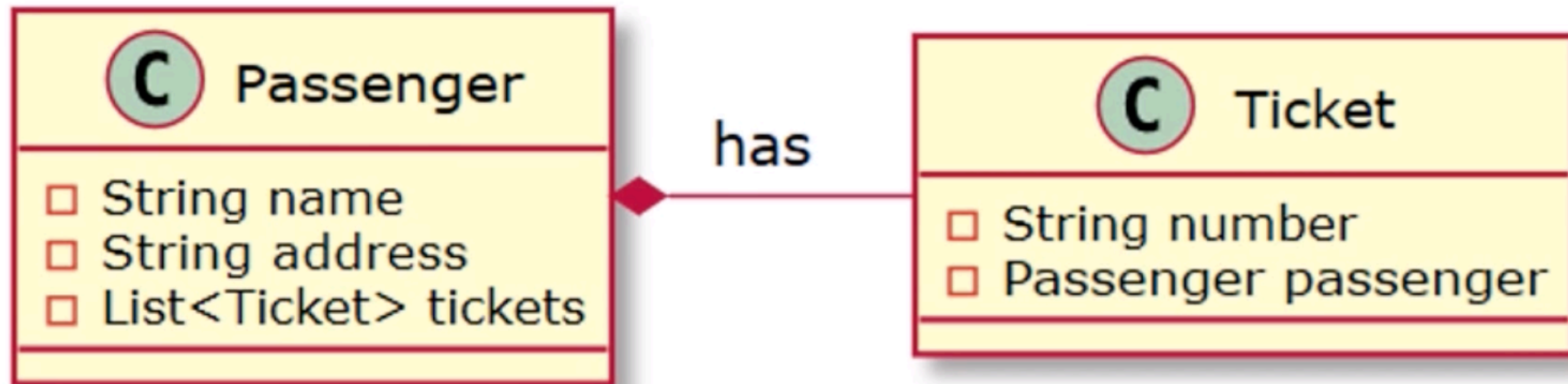
Inconvénients

- Courbe d'apprentissage
- Plus difficile à débbugger
- Peut avoir des problèmes de performance
- JDBC est plus proche de la base de donnée, on a donc un meilleur contrôle de ce qu'on veut faire.
- Si on a besoin de la fonctionnalité spécifique d'une base de donnée (ex: triggers, window functions, ...)

Objet-Relationnel 'mismatch'

- Les modèles objets et les base de données relationnels ne sont pas faits pour s'entendre.
- Objets interconnectés vs tables relationnelles
- Granularité, héritage, identité, associations et navigation

Granularité



Granularité

```
public class Passenger {  
    private String name;  
    private String address;  
    private List<Ticket> tickets;  
}
```

```
public class Ticket {  
    private String number;  
    private Passenger passenger;  
}
```

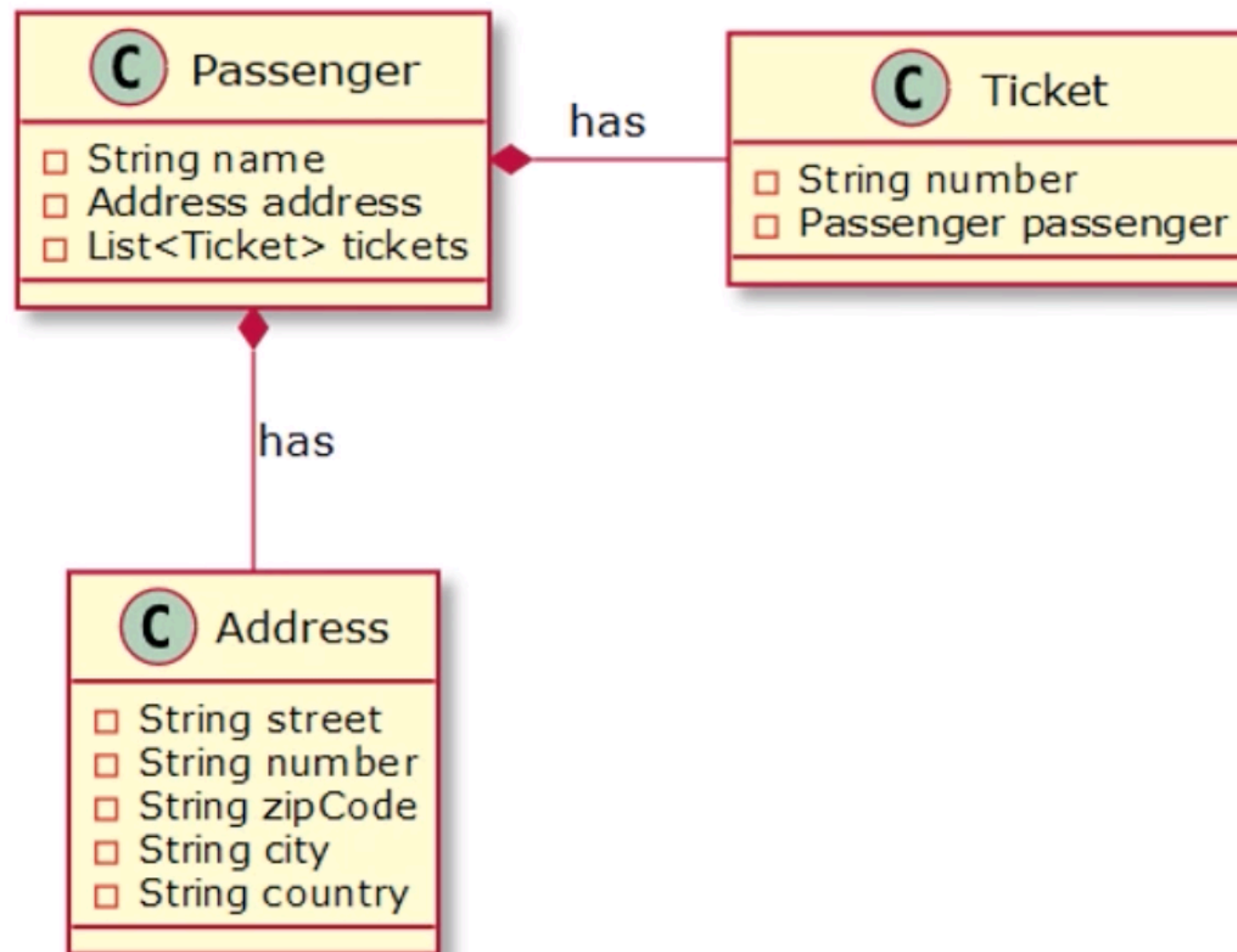
Granularité

```
create table PASSENGERS (  
    NAME varchar(255),  
    ADDRESS varchar(255),  
    primary key (NAME)  
)  
  
create table TICKETS (  
    NUMBER varchar(255),  
    PASSENGER_NAME varchar(255),  
    primary key (NUMBER)  
)  
  
alter table TICKETS  
    add constraint FK_PASSENGERS  
    foreign key (PASSENGER_NAME)  
    references PASSENGERS (NAME)
```

Granularité

- Qu'arrive-t'il avec un ORM et la BD quand on rajoute une abstraction comme la suivante ?

Granularité



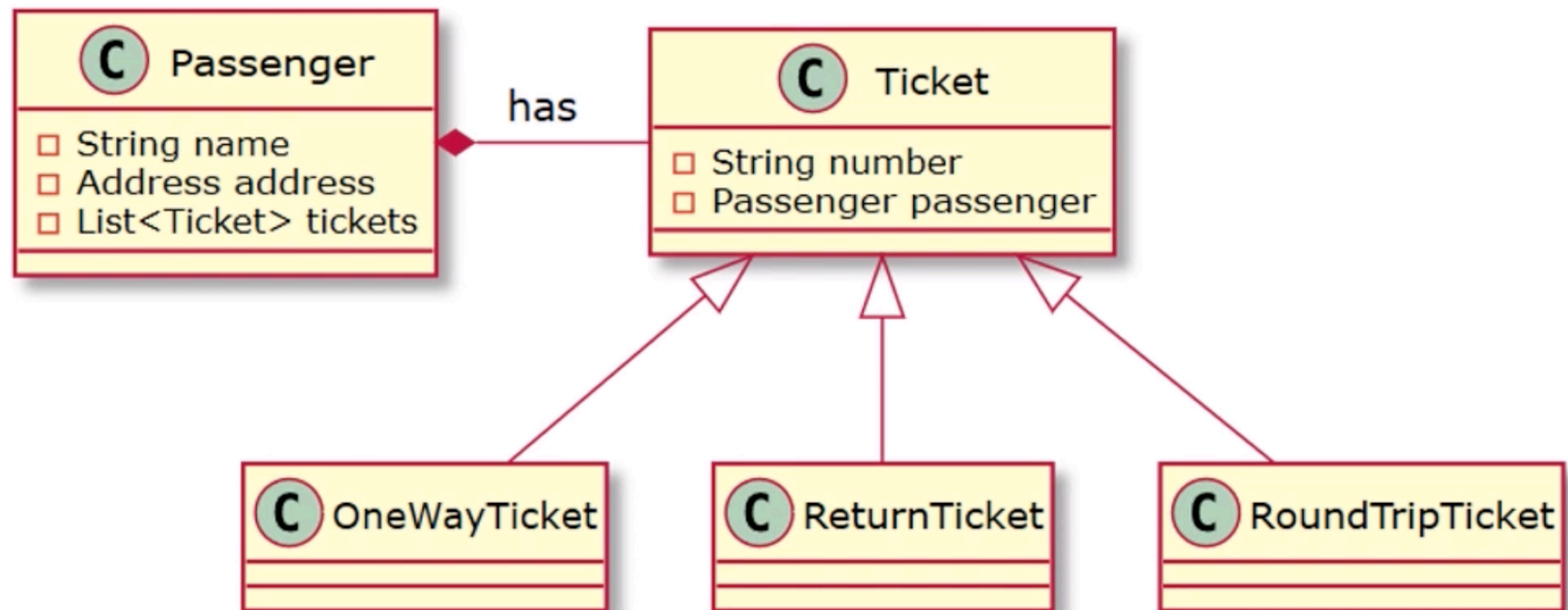
Granularité

- Les fournisseurs de BD ne nous permettent pas de définir des 'types SQL'. Il faut donc recourir à d'autres stratégies pour 'mapper' une abstraction en SQL.
- La diapo suivant illustre une des stratégie

Granularité

```
create table PASSENGERS (  
    NAME varchar(255),  
    ADDRESS_STREET varchar(30),  
    ADDRESS_NUMBER varchar(6),  
    ADDRESS_ZIPCODE varchar(10),  
    ADDRESS_CITY varchar(25),  
    ADDRESS_COUNTRY varchar(25),  
    primary key (NAME)  
)
```

Héritage



Notion d'unicité

Clés primaires (PRIMARY KEY) dans une BD

- Soit un champ spécial ('surrogate') pour la clé primaire ou soit un ou plusieurs champs du domaine d'application (ex: nom, prénom).
- Aujourd'hui, on utilise presque exclusivement un champ spécial

Notion d'unicité

- Égalité d'objet (==). Deux références pointent sur le même objet
- Égalité logique (.equals)
- Dans la bd, l'unicité est identifiée par la clé primaire
- Rien dans Java permet d'égaliser la notion d'unicité des bases de données relationnelles.

Notion d'unicité

- Dans la diapo suivante, on utilise un champ de la table comme clé primaire. Notamment le nom dans la table 'passenger'
- Cela va occasionner des problèmes si l'on doit modifier le nom

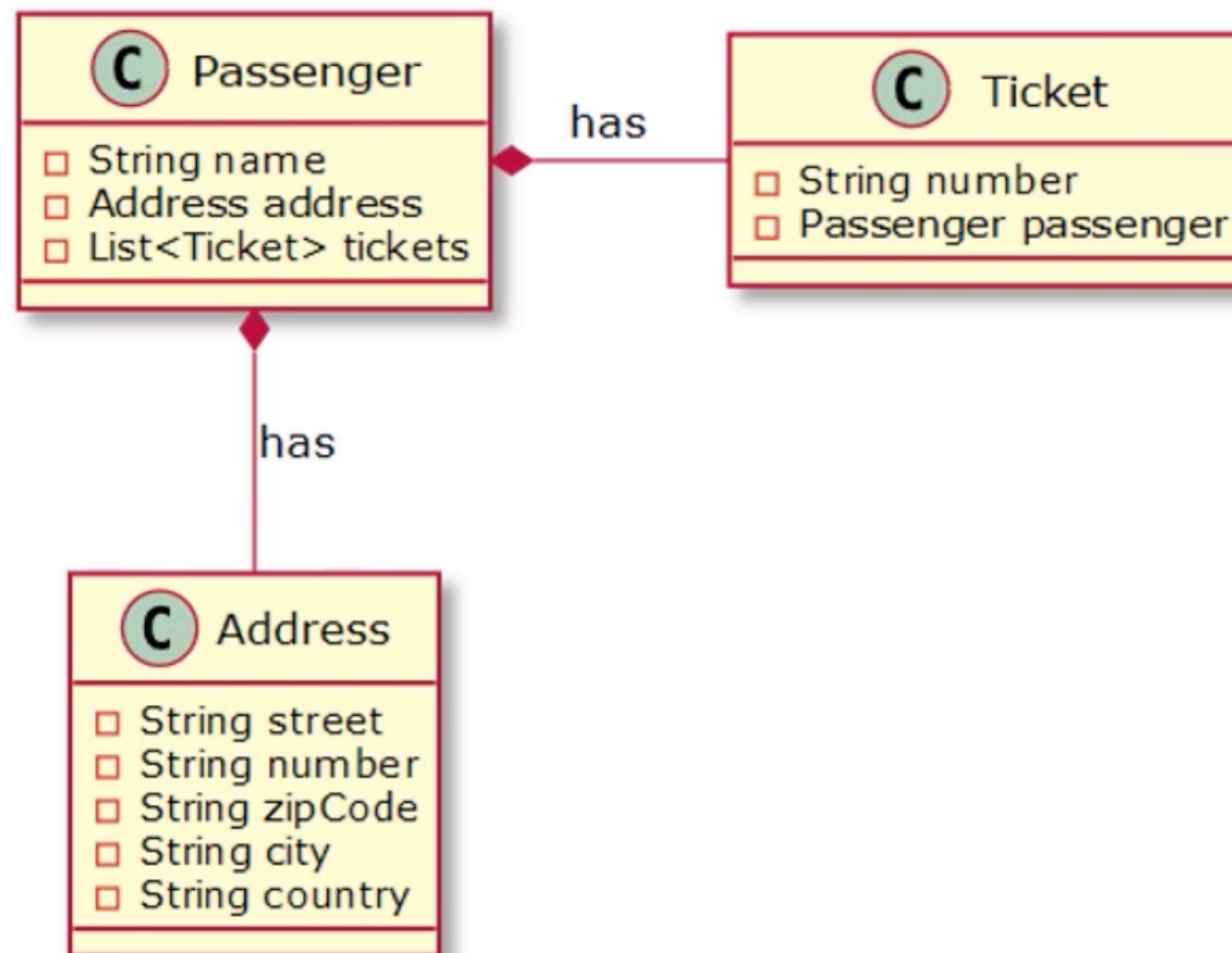
Identité - clé 'surrogate' vs 'sane'

```
create table PASSENGERS (  
    NAME varchar(255),  
    ADDRESS varchar(255),  
    primary key (NAME)  
)  
  
create table TICKETS (  
    NUMBER varchar(255),  
    PASSENGER_NAME varchar(255),  
    primary key (NUMBER)  
)  
  
alter table TICKETS  
    add constraint FK_PASSENGERS  
    foreign key (PASSENGER_NAME)  
    references PASSENGERS (NAME)
```


Identité - clé 'surrogate' vs 'sane'

```
create table PASSENGERS (  
    ID integer not null,  
    NAME varchar(255),  
    primary key (ID)  
)  
  
create table TICKETS (  
    ID integer not null,  
    NUMBER varchar(255),  
    PASSENGER_ID integer,  
    primary key (ID)  
)  
  
alter table TICKETS  
    add constraint FK_PASSENGERS  
    foreign key (PASSENGER_ID)  
    references PASSENGERS (ID)
```


Association dans le modèle objet



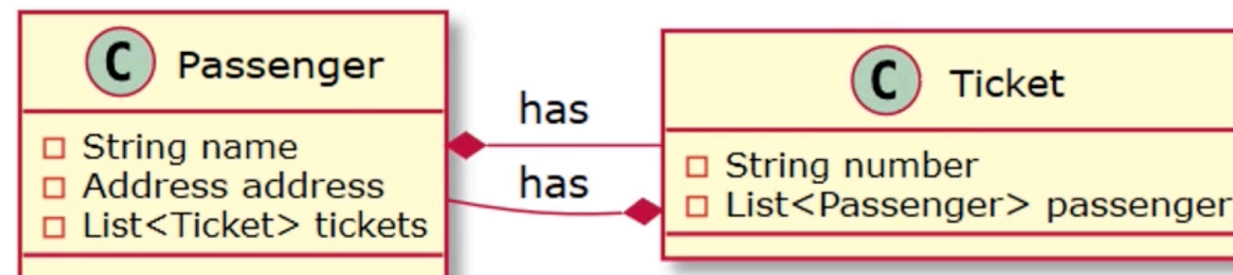
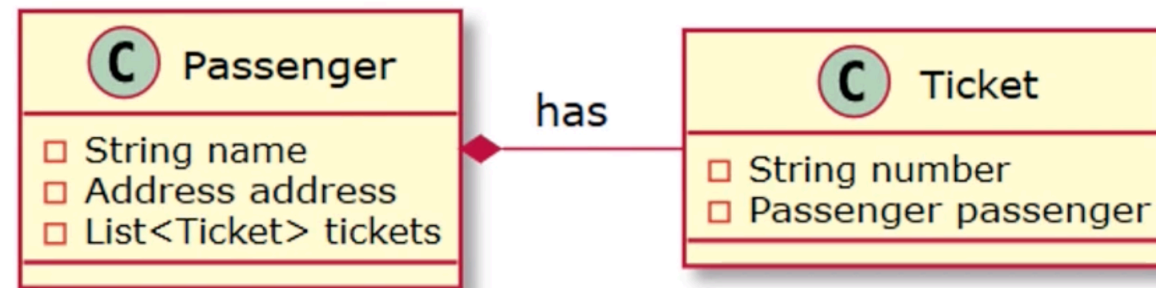
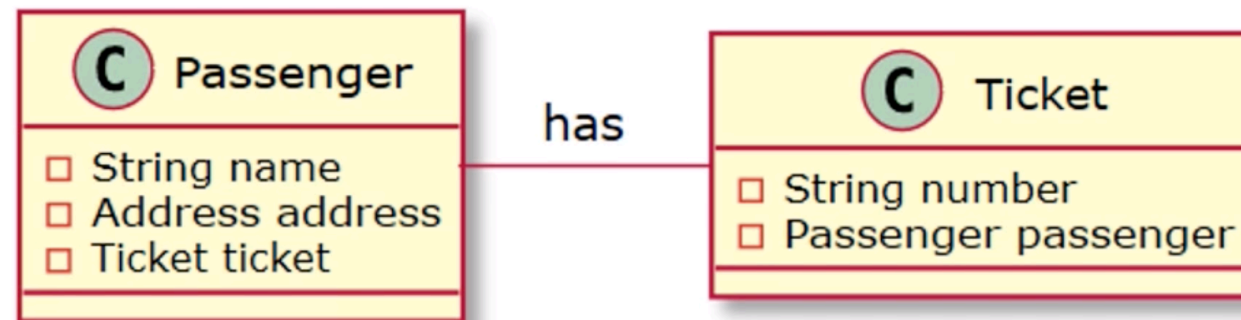
Association dans le modèle relationnel

```
create table PASSENGERS (  
    ID integer not null,  
    NAME varchar(255),  
    ADDRESS_STREET varchar(30),  
    ADDRESS_NUMBER varchar(6),  
    ADDRESS_ZIPCODE varchar(10),  
    ADDRESS_CITY varchar(25),  
    ADDRESS_COUNTRY varchar(25),  
    primary key (ID)  
)  
  
create table TICKETS (  
    ID integer not null,  
    NUMBER varchar(255),  
    PASSENGER_ID integer,  
    primary key (ID)  
)
```

Associations

- Modèle objet
 - Associations sont directionnelles. Elles suivent la direction du 'has' (ou 'has-a')
 - one-to-one
 - one-to-many
 - many-to-many
- Modèle relationnel
 - Associations sont toujours bi-directionnelles
 - Toujours many-to-one

Associations directionnelles en java



Associations

```
create table PASSENGERS_TICKETS (  
    PASSENGER_ID integer not null,  
    TICKET_ID integer not null,  
    primary key (PASSENGER_ID, TICKET_ID)  
)
```

```
alter table PASSENGERS_TICKETS  
    add constraint FK_PASSENGERS  
    foreign key (PASSENGER_ID)  
    references PASSENGERS (ID)
```

```
alter table PASSENGERS_TICKETS  
    add constraint FK_TICKETS  
    foreign key (TICKET_ID)  
    references TICKETS (ID)
```

Navigation

```
for(Ticket ticket: passenger.getTickets())
```

```
SELECT * FROM PASSENGERS WHERE ID = 727423
```

```
SELECT * FROM PASSENGERS, TICKETS  
WHERE PASSENGERS.ID = 727423 AND  
PASSENGERS.ID = TICKETS.PASSENGER_ID
```

@Entity

- Une classe annotée avec l'annotation @Entity définit une classe qui sera persistée dans une base de données relationnelle.
- La classe doit être 'top level'
- Doit avoir un constructeur public ou protected sans argument (obligatoire)
- Classes 'finales' ne sont pas recommandées. Hibernate peut utiliser un 'proxy' pour les classes annotées @Entity

@Entity

- héritage
- associations polymorphiques
- 'query' polymorphiques

@Entity - ce qu'on peut persister

- Types primitifs
- D'autres @Entity
- Enums
- Types 'embedable'
- Types 'serializables'
- Collections

@Table

- Permet de spécifier le nom de la table dans la base de donnée relationnelle.

Entity access type

- Field base

```
@Entity
@Table(name = "PASSENGERS")
public class Passenger {
    @Id
    @Column(name = "PASSENGER_ID")
    private int id;

    @Column(name = "PASSENGER_NAME", table = "PASSENGERS")
    private String name;
```

Entity access type

- Property base

```
@Entity
@Table(name = "PASSENGERS")
public class Passenger {
    private int id;

    @Id
    @Column(name = "PASSENGER_ID")
    public int getId() {
        return id;
    }
}
```

Mixed access types

```
@Entity
@Table(name = "PASSENGERS")
@Access(AccessType.FIELD)
public class Passenger {
    @Id
    private int id;
    private String name;

    public int getId() {
        return id;
    }

    @Access(AccessType.PROPERTY)
    public String getName() {
        return name;
    }
}
```

Access Types - Pourquoi ?

- Field base : Permet d'omettre les getter pour les champs que l'on ne veut pas exposer.
- Field base : Meilleure lisibilité du code
- Property base : Permet d'inclure une logique additionnelle en l'incorporant dans le getter/setter

Primary Key (clé primaire)

- Toute @Entity doit avoir une clé primaire
- Une clé primaire peut être un seul champ ou plusieurs champs. On ne verra que les clés à un champ dans ce cours
- @GeneratedValue
 - Déterminer la stratégie de génération de clé
 - Utilisé en conjonction avec @Id
 - Peut être utilisé sur un champ persistant

Clé primaire

@GeneratedValue

```
@Entity
@Table(name = "TICKETS")
public class Ticket {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private int id;
}
```


Clé primaire @GeneratedValue

```
public class Main {  
  
    public static void main(String[] args) {  
        EntityManagerFactory emf = Persistence.createEntityManagerFactory(  
            EntityManager em = emf.createEntityManager();  
  
        em.getTransaction().begin();  
  
        Ticket ticket1 = new Ticket();  
        ticket1.setSeries("AA");  
        ticket1.setNumber("1234");  
        ticket1.setOrigin("Bucharest");  
        ticket1.setDestination("London");  
        em.persist(ticket1);  
  
        em.getTransaction().commit();  
        emf.close();  
    }  
}
```

Clé primaire @GeneratedValue

Hibernate:

```
create table hibernate_sequence (  
    next_val bigint  
) engine=InnoDB
```

Hibernate:

```
select  
    next_val as id_val  
from  
    hibernate_sequence for update
```

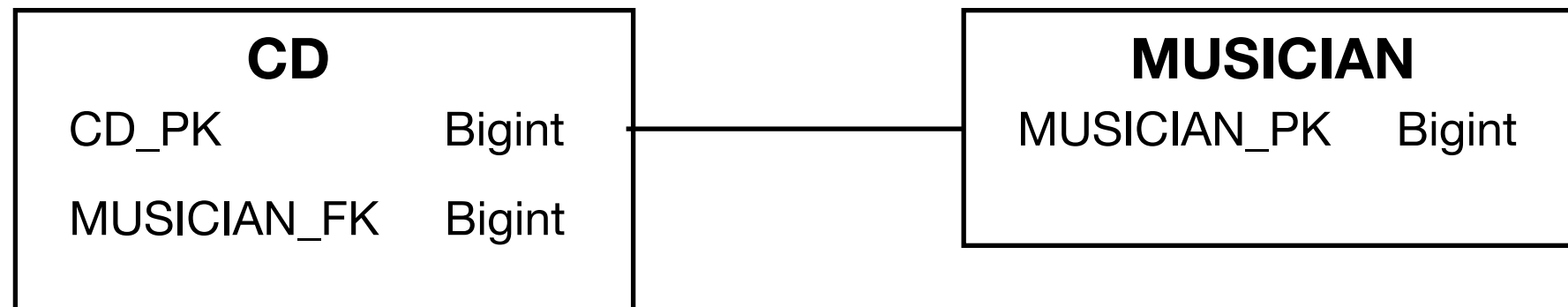
Hibernate:

```
update  
    hibernate_sequence  
set  
    next_val= ?  
where  
    next_val=?
```

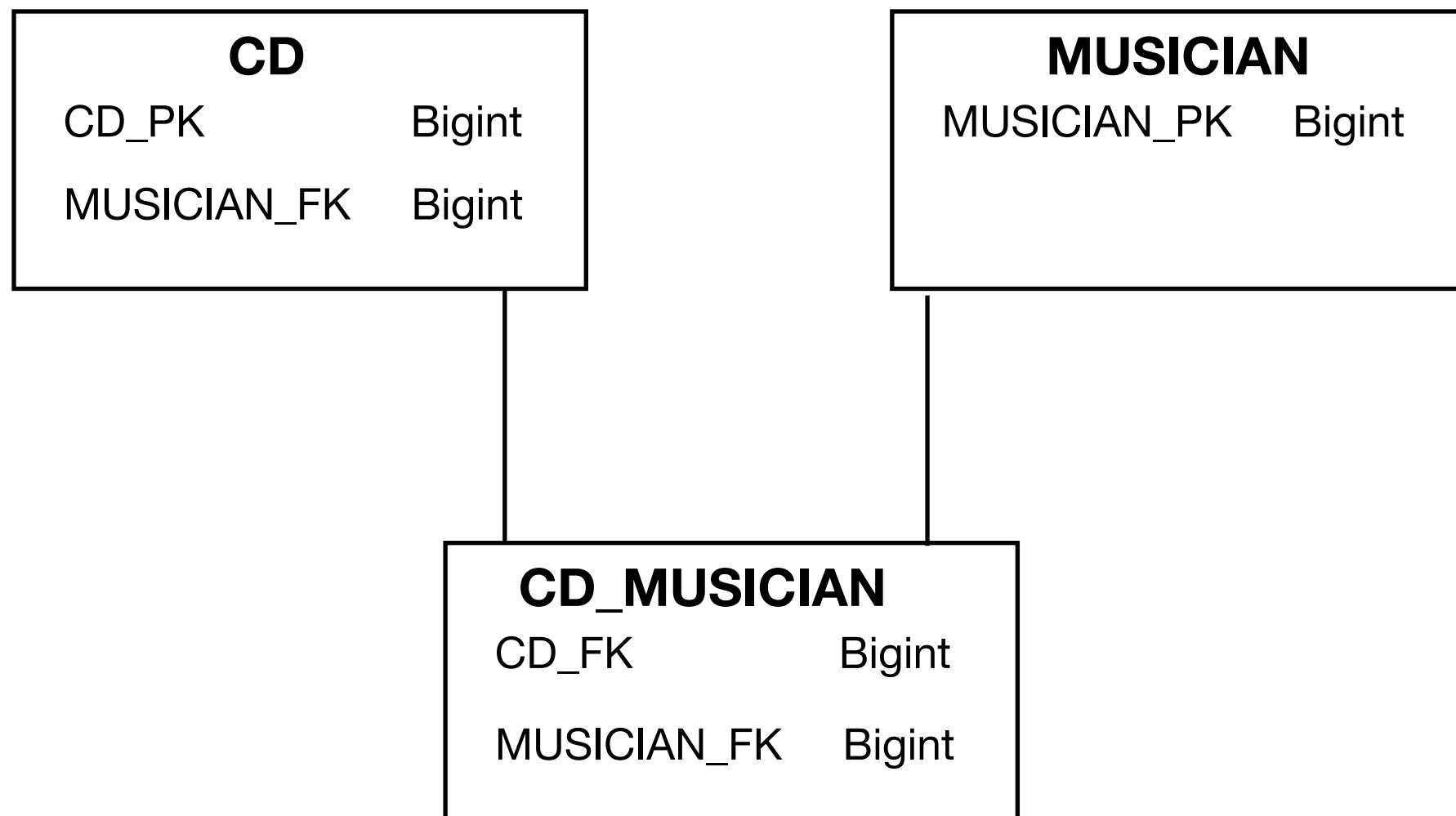
Associations en RDBMS

- Collection d'associations (relation en anglais)
- Table
- Primary Key
- Join Column
- Join Table

Associations en RDBMS



Associations en RDBMS



Associations - JPA défauts

- | | |
|----------------|-------------|
| • One-to-one | Join Column |
| • Many-to-one | Join Column |
| • One-to-many | Join Table |
| • Many-to-many | Join Table |

Relation - Direction

- Unidirectionnelle
- Bi-directionnelle
- Toute relation a un côté qui détient la relation (owner side)
- Le côté qui détient la relation (owner) est celui qui fait les mises à jour à la bd
- Le côté 'plusieurs ou many' est toujours le côté qui détient la relation (owner)

Relations - annotations

- @OneToMany (mappedBy = `""`)
- @ManyToOne - @JoinColumn(name = `""`)
- @ManyToMany
- @OneToOne

OneToMany Passenger.java

```
@GeneratedValue
@Column(name = "PASSENGER_ID")
private int id;

@Column(name = "PASSENGER_NAME")
private String name;
```

```
1
@OneToMany(mappedBy = "passenger")
private List<Ticket> tickets = new ArrayList<>();

public List<Ticket> getTickets() {
    return Collections.unmodifiableList(tickets);
}
```

OneToMany

Ticket.java et Main.java

```
@ManyToOne
@JoinColumn(name = "PASSENGER_ID")
private Passenger passenger;
```

```
public static void main(String[] args) {
    EntityManagerFactory emf = Persistence.createEntityManagerFactory("...");
    EntityManager em = emf.createEntityManager();

    em.getTransaction().begin();

    Passenger john = new Passenger( name: "John Smith");
    Ticket ticket1 = new Ticket( number: "AA1234");
    ticket1.setPassenger(john);

    Ticket ticket2 = new Ticket( number: "BB5678");
    ticket2.setPassenger(john);

    john.addTicket(ticket1);
    john.addTicket(ticket2);

    em.persist(john);

    em.persist(ticket1);
    em.persist(ticket2);
}
```

ManyToMany

```
@Entity
@Table(name = "PASSENGERS")
public class Passenger {

    @Id
    @GeneratedValue
    @Column(name = "PASSENGER_ID")
    private int id;

    @Column(name = "PASSENGER_NAME")
    private String name;

    @ManyToMany(mappedBy = "passengers")
    private List<Ticket> tickets = new ArrayList<>();

    public Passenger(String name) { this.name = name; }

    public Passenger() {

    }

}
```

ManyToMany

```
@Entity
@Table(name = "TICKETS")
public class Ticket {
    @Id
    @GeneratedValue
    @Column(name = "ID")
    private int id;

    @Column(name = "NUMBER")
    private String number;

    @ManyToMany
    private List<Passenger> passengers = new ArrayList<>();

    public Ticket(String number) { this.number = number; }
```

ManyToMany

```
Passenger john = new Passenger( name: "John Smith");  
Passenger mike = new Passenger( name: "Michael Johnson");
```

```
Ticket ticket1 = new Ticket( number: "AA1234");  
Ticket ticket2 = new Ticket( number: "BB5678");
```

```
john.addTicket(ticket1);  
john.addTicket(ticket2);  
mike.addTicket(ticket1);  
mike.addTicket(ticket2);
```

```
ticket1.addPassenger(john);  
ticket1.addPassenger(mike);  
ticket2.addPassenger(john);  
ticket2.addPassenger(mike);
```

Cascades

Nous avons vu jusqu'à présent que l'on doit sauver individuellement chacune des entités comme illustré ci-dessous.

```
em.persist(airport);  
em.persist(passenger1);  
em.persist(passenger2);
```

```
em.persist(ticket1);  
em.persist(ticket2);  
em.persist(ticket3);
```

```
em.getTransaction().commit();
```

Cascades

Mais en ajoutant une directive de cascade au bon endroit, il est possible de demander à JPA de sauvegarder pour nous les associations d'une entité comme illustré ci-dessous.

2 usages

```
@OneToMany(mappedBy = "passenger", cascade = CascadeType.PERSIST)  
private List<Ticket> tickets = new ArrayList<>();
```

```
em.persist(airport);  
em.persist(passenger1);  
em.persist(passenger2);  
|  
// em.persist(ticket1);  
// em.persist(ticket2);  
// em.persist(ticket3);
```


Cascades

Voici les différentes options de cascades disponibles

```
i f CascadeType.ALL (javax.persistence)
f CascadeType.DETACH (javax.persistence)
f CascadeType.MERGE (javax.persistence)
E CascadeType javax.persistence
f CascadeType.PERSIST (javax.persistence)
f CascadeType.REFRESH (javax.persistence)
f CascadeType.REMOVE (javax.persistence)
```

L'option CascadeType.ALL permet de gérer tous les cas, soit PERSIST, MERGE et REMOVE

2 usages

```
⚠ @OneToMany(mappedBy = "passenger", cascade = CascadeType.ALL)
private List<Ticket> tickets = new ArrayList<>();
```


Classes 'Embeddable'

- Classes n'ayant pas d'identité
- Ne peuvent être partagées entre plusieurs entités
- N'appartiennent qu'à une seule entité
- Généralement, de très petite granularité. Peut être utilisé pour représenter un état d'une entité.

Classes 'Embeddable'

- @Embeddable
- @AttributeOverride: Permet de modifier une des propriétés définissant la classe embeddable
- @AttributeOverrides: Comme attribute override mais permet de modifier plusieurs propriétés définissant la classe embeddable
- On peut également avoir des collections associées à une entité à l'aide des annotation @ElementCollection et @CollectionTable. En dehors de ce cours

Classes 'Embeddable'

```
@Embeddable|
public class Address {
    private String street;
    private String number;
    private String zipCode;
    private String city;
```

Dans la classe 'Passenger'

```
@Embedded
@AttributeOverrides({
    @AttributeOverride(name = "street", column = @Column(name = "PASSENGER_STREET")),
    @AttributeOverride(name = "number", column = @Column(name = "PASSENGER_NUMBER")),
    @AttributeOverride(name = "zipCode", column = @Column(name = "PASSENGER_ZIP_CODE")),
    @AttributeOverride(name = "city", column = @Column(name = "PASSENGER_CITY"))| I
})
private Address address;
```

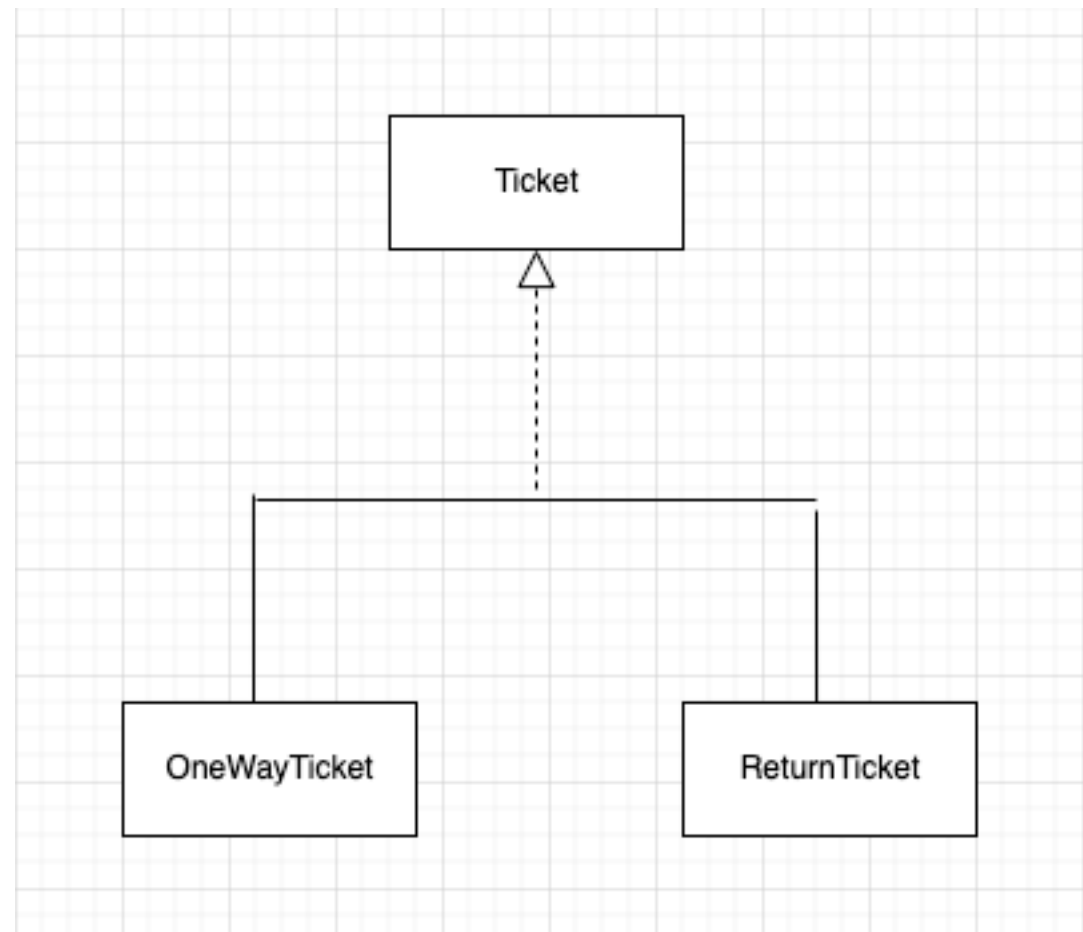
Classes 'Embeddable'

```
public class Main {  
    public static void main(String[] args) {  
        EntityManagerFactory emf =  
            Persistence.createEntityManagerFactory("persistenceUnitName");  
        EntityManager em = emf.createEntityManager();  
  
        em.getTransaction().begin();  
        Passenger john = new Passenger("John Smith");  
        Address address = new Address();  
  
        address.setStreet("Flowers Street");  
        address.setNumber("3");  
        address.setZipCode("012345");  
        address.setCity("Boston");  
        john.setAddress(address);  
  
        em.persist(john);  
    }  
}
```

Héritage des entités

- Héritage des entités et des non-entités
- Différentes stratégies de 'mapping'

Héritage des entités



Héritage des entités

```
public abstract class Ticket {  
  
    private int id;  
    private String number;  
  
    public int getId() { return id; }  
  
    public String getNumber() { return number; }  
  
    public void setNumber(String number) { this.number = number; }  
}
```


Héritage des entités

```
import java.time.LocalDate;
```

```
public class OneWayTicket extends Ticket {  
  
    private LocalDate latestDepartureDate;  
  
    public LocalDate getLatestDepartureDate() { return latestDepartureDate; }  
  
    public void setLatestDepartureDate(LocalDate latestDepartureDate) {  
        this.latestDepartureDate = latestDepartureDate;  
    }  
}
```


Héritage des entités

```
import java.time.LocalDate;

public class ReturnTicket extends Ticket {

    private LocalDate latestReturnDate;

    public LocalDate getLatestReturnDate() { return latestReturnDate; }

    public void setLatestReturnDate(LocalDate latestReturnDate) {
        this.latestReturnDate = latestReturnDate;
    }
}
```

Héritage des entités

```
@Entity
@Table(name = "TICKETS")
public abstract class Ticket {
    @Id
    @GeneratedValue
    private int id;
    private String number;

    public int getId() { return id; }

    public String getNumber() { return number; }

    public void setNumber(String number) { this.number = number; }
}
```

Héritage des entités

```
@Entity
public class OneWayTicket extends Ticket {

    private LocalDate latestDepartureDate;

    public LocalDate getLatestDepartureDate() { return latestDepartureDate; }

    public void setLatestDepartureDate(LocalDate latestDepartureDate) {
        this.latestDepartureDate = latestDepartureDate;
    }
}
```

Héritage des entités

```
@Entity|
public class ReturnTicket extends Ticket {

    private LocalDate latestReturnDate;

    public LocalDate getLatestReturnDate() { return latestReturnDate; }

    public void setLatestReturnDate(LocalDate latestReturnDate) {
        this.latestReturnDate = latestReturnDate;
    }
}
```

Héritage des entités

```
public class Main {  
  
    public static void main(String[] args) {  
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("h  
        EntityManager em = emf.createEntityManager();  
  
        em.getTransaction().begin();  
  
        OneWayTicket oneWayTicket = new OneWayTicket();  
        oneWayTicket.setNumber("AA1234");  
        oneWayTicket.setLatestDepartureDate(LocalDate.of(year: 2021, month: 3, dayOfMonth: 20));  
        ReturnTicket returnTicket = new ReturnTicket();  
        returnTicket.setNumber("BB5678");  
        returnTicket.setLatestReturnDate(LocalDate.of(year: 2022, month: 1, dayOfMonth: 31));  
  
        em.persist(oneWayTicket);  
        em.persist(returnTicket);  
    }  
}
```

Stratégie de mapping d'héritage

- Single Table per class
- Joined Subclass
- Table per Concrete class

Stratégie de mapping d'héritage

- Toutes les classes de la hiérarchie sont mappées sous une table
- L'utilisation d'un discriminateur est utilisé pour permettre à JPA de déterminer le type d'instance est stocké dans la table
- Parfait pour l'utilisation de relations polymorphiques
- Requiert que les colonnes soient 'nullable'

Stratégie de mapping d'héritage

- @Inheritance
- @DiscriminatorColumn
- @DiscriminatorValue

Stratégies de mapping d'héritage

```
@Entity
@Table(name = "TICKETS")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "TICKET_TYPE")
public abstract class Ticket {
    @Id
    @GeneratedValue
    private int id;
    private String number;

    public int getId() { return id; }

    public String getNumber() { return number; }

    public void setNumber(String number) { this.number = number; }
}
```

Stratégies de mapping d'héritage

```
@Entity
@DiscriminatorValue("0")
public class OneWayTicket extends Ticket {

    private LocalDate latestDepartureDate;

    public LocalDate getLatestDepartureDate() {
        return latestDepartureDate;
    }

    public void setLatestDepartureDate(LocalDate latestDepartureDate) {
        this.latestDepartureDate = latestDepartureDate;
    }
}
```

Stratégies de mapping d'héritage

```
@Entity
@DiscriminatorValue("R")
public class ReturnTicket extends Ticket {

    private LocalDate latestReturnDate;

    public LocalDate getLatestReturnDate() {
        return latestReturnDate;
    }

    public void setLatestReturnDate(LocalDate latestReturnDate) {
        this.latestReturnDate = latestReturnDate;
    }
}
```

Stratégies de mapping d'héritage

```
public class Main {  
  
    public static void main(String[] args) {  
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("h.  
        EntityManager em = emf.createEntityManager();  
  
        em.getTransaction().begin();  
  
        OneWayTicket oneWayTicket = new OneWayTicket();  
        oneWayTicket.setNumber("AA1234");  
        oneWayTicket.setLatestDepartureDate(LocalDate.of(year: 2021, month: 3, dayOfMonth: 20));  
  
        ReturnTicket returnTicket = new ReturnTicket();  
        returnTicket.setNumber("BB5678");  
        returnTicket.setLatestReturnDate(LocalDate.of(year: 2022, month: 1, dayOfMonth: 31));  
  
        em.persist(oneWayTicket);  
        em.persist(returnTicket);  
    }  
}
```

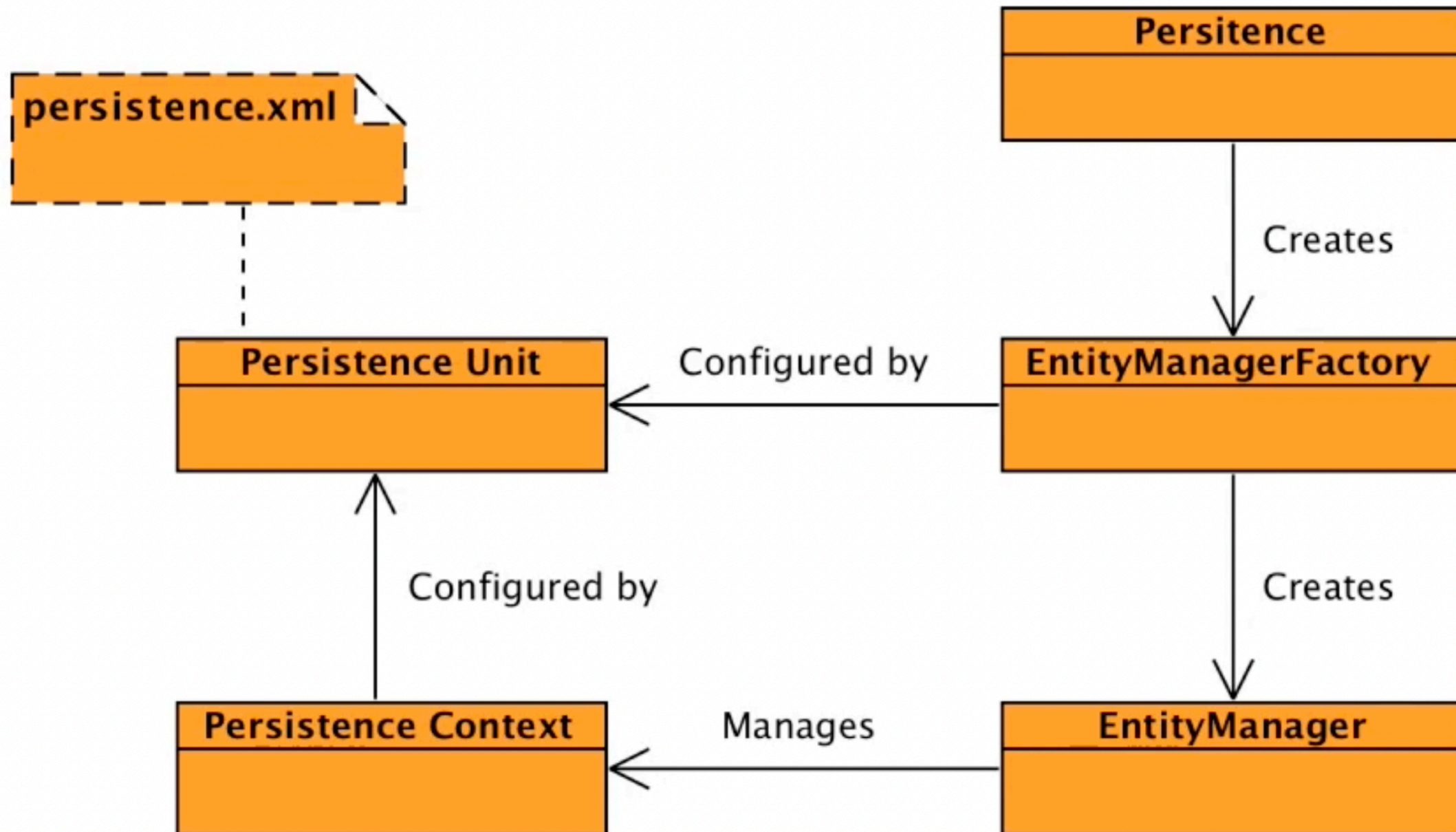
Entity Manager

- Interfaces et classes pour la persistance
- Méthodes de l'interface EntityManager
- Les 'états (ou states)
- Commit et flushs
- EntityManager de type Application et Container

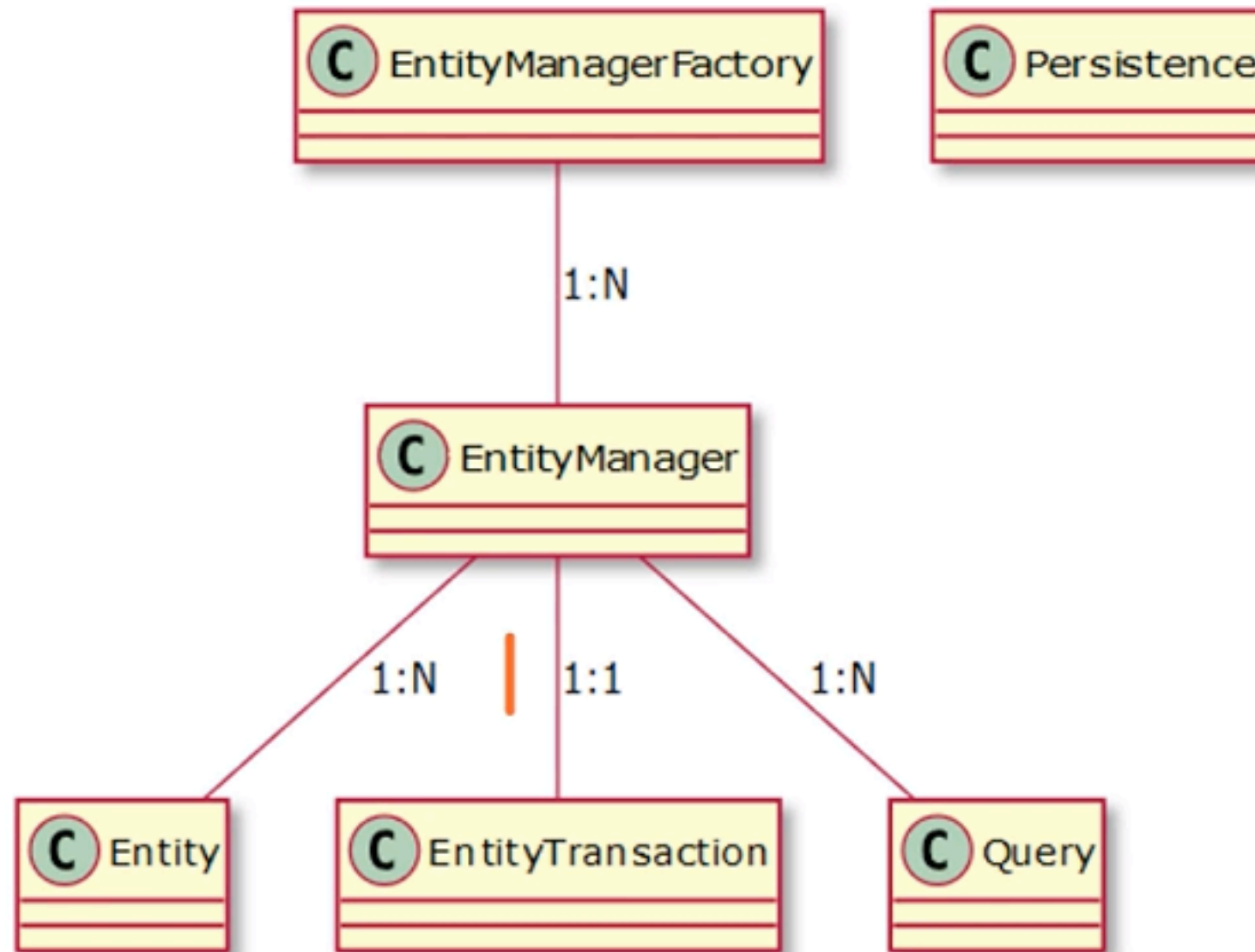
EntityManager

- Une interface permettant d'interagir avec le 'persistence context'
- persistence context: L'ensemble des 'entités' gérées par le persistence context. Pour chaque entité, il existe une et une seule instance dans le persistence context. Celui-ci gère l'état des entités qu'il contient.
- Persistence Unit : L'ensemble des classes gérées par le EntityManager. Chacune de ces classes peuvent se trouver dans le persistence context.

Entity Manager



Entity Manager



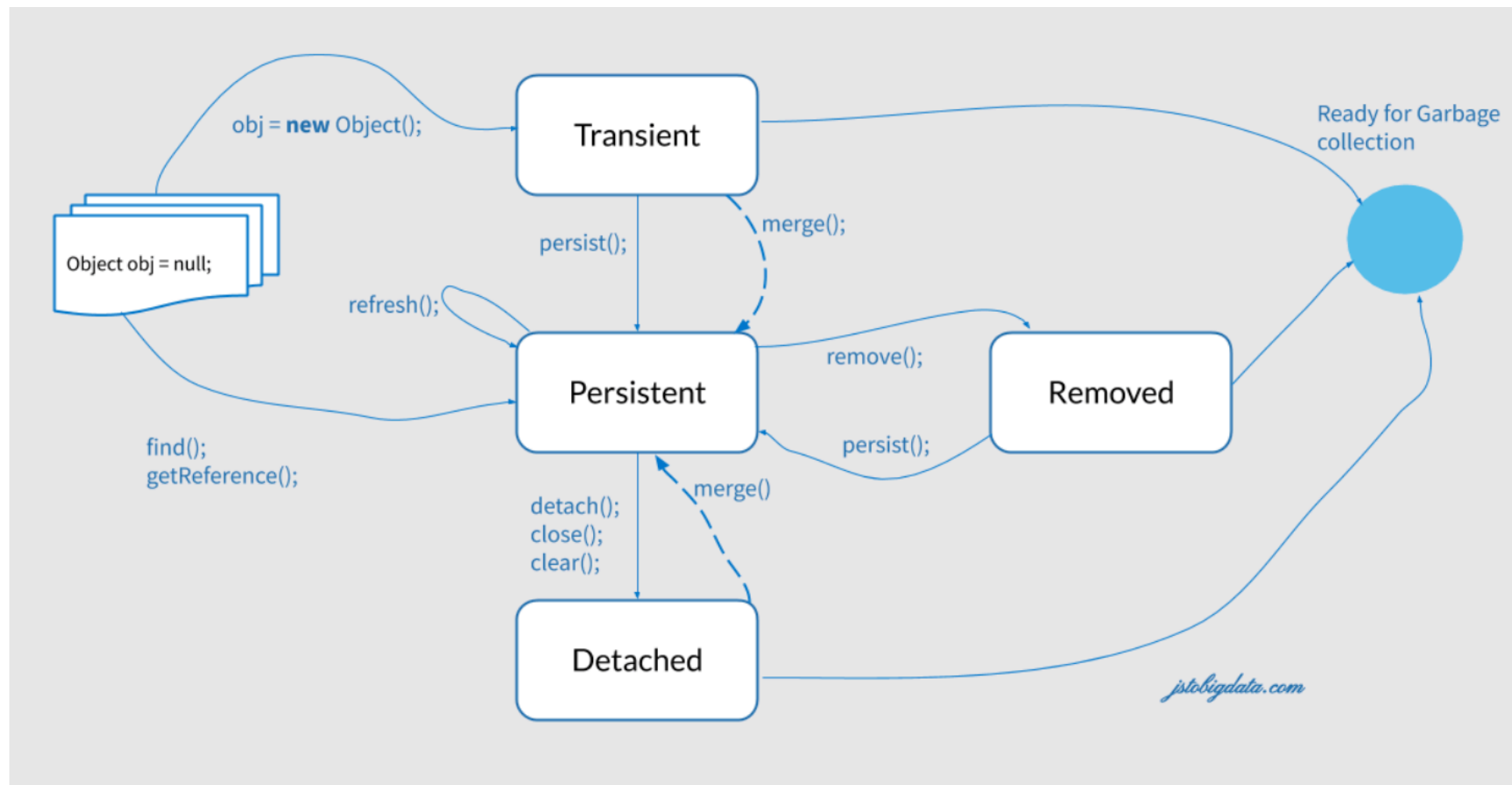
Entity States ou État des entités

- New/Transient
- Managed/Persisted
- Removed
- Detached

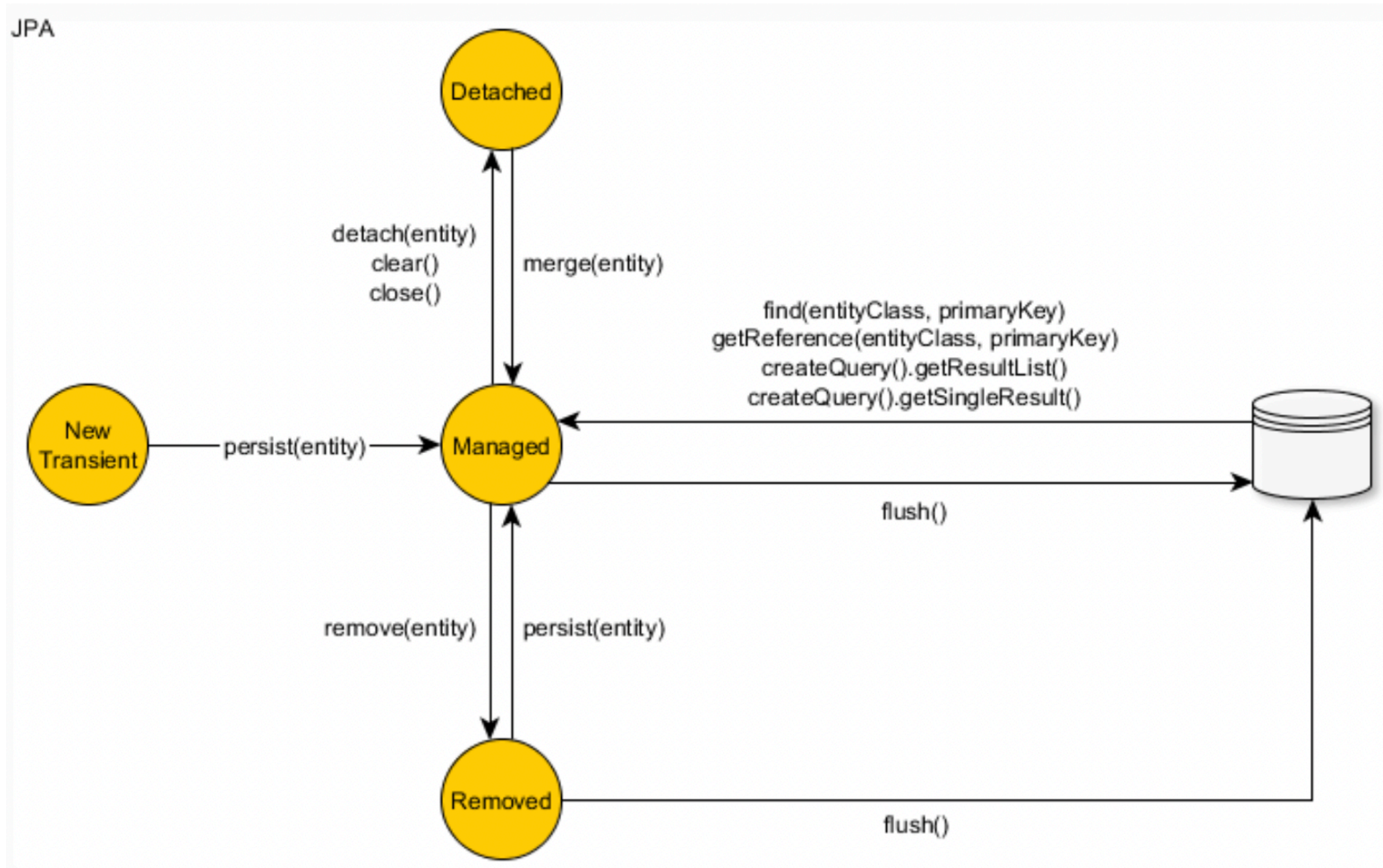
Entity Manager méthodes

- `persist()` : Sauve une nouvelle instance dans la bd
- `remove()` : Enlève une instance de la bd (delete from...)
- `merge()` : Réattache au persistant context une instance qui est 'detached'
- `find()` : Hydrate une instance à partir de la bd.
- `lock()` : Obtient un lock de la bd ce qui a pour effet d'empêcher d'autres utilisateurs de mettre à jour une Entité.
- `detach()` : Enlève une instance du persistant context
- `refresh()` : Resynchronise avec la bd une instance qui est déjà 'managed/persistent'

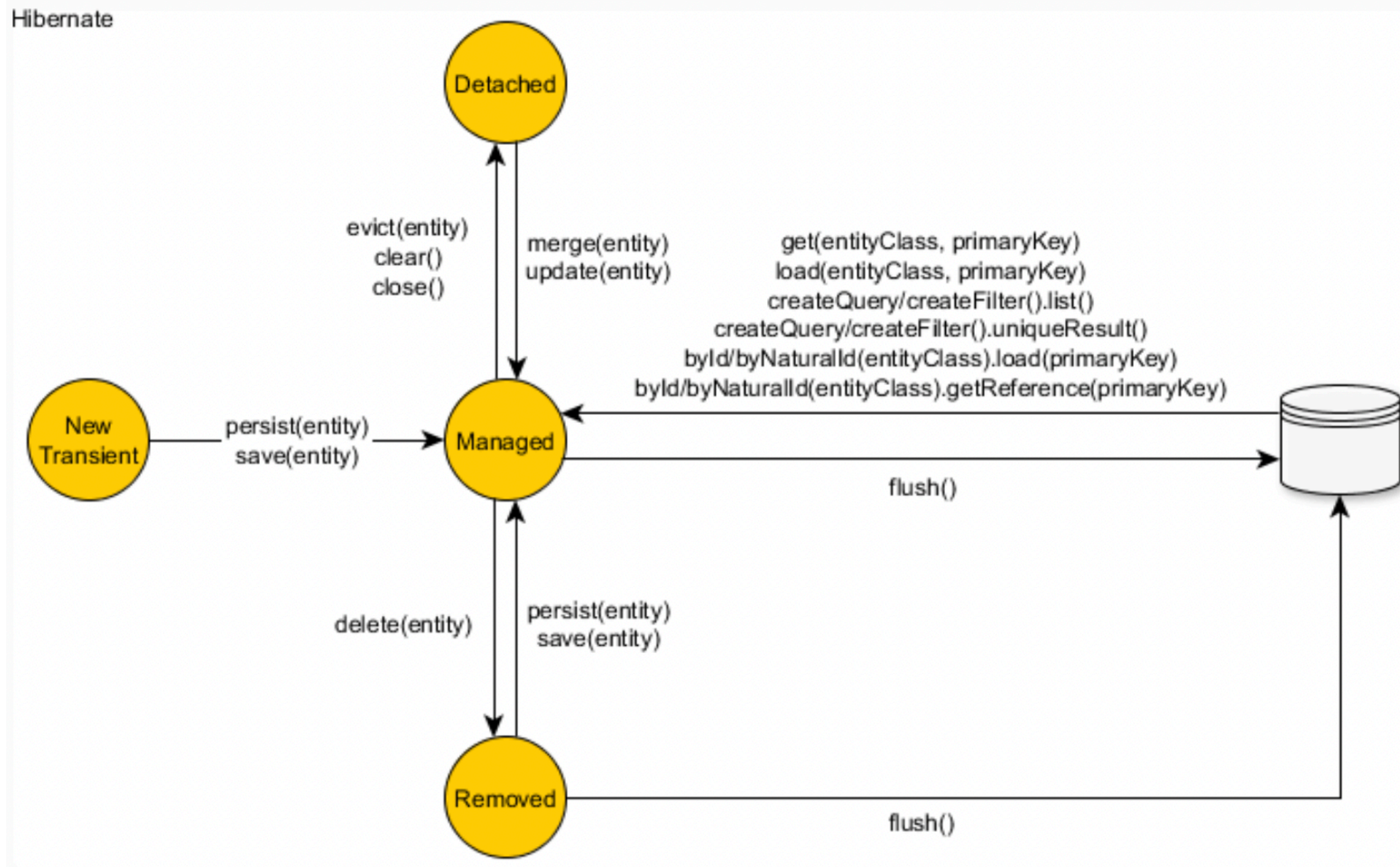
Entity states



Entity states



Entity states



New/Transient

- Un objet fraîchement créé et qui n'a jamais été associé à un 'persistence context' aura le state 'New ou Transient'

```
1. Student student = new Student("email@dot.com");
```

Persistent/Managed

- Tout objet associé au 'persistence context' est dans l'état 'persistent ou managed'. Tout changement fait à un objet dans cet état est automatiquement propagé à la bd sans avoir besoin d'invoquer une commande de sauvegarde (ex: em.persist, em.merge). Les changements sont propagés au tx.commit()

```
1. Student student = new Student("test@email.com");
2.
3. // Make changes and see if the data is updated automatically
4. EntityManager entityManager = entityManagerFactory.createEntityManager();
5. EntityTransaction tx = entityManager.getTransaction();
6. tx.begin();
7. entityManager.persist(student);
8. //purposely made changes - did not manually update
9. student.setEmail("updated_email@gmail.com");
10. Long persistedId = student.getId();
11. tx.commit();
12. entityManager.close();
```


Persistent/Managed

- Pour vérifier le code précédent, on peut exécuter le code suivant.

```
1. // Test if the email address was updated
2. entityManager = entityManagerFactory.createEntityManager();
3. tx = entityManager.getTransaction();
4. tx.begin();
5. student = entityManager.find(Student.class, persistedId);
6. tx.commit();
7. entityManager.close();
8. System.out.println("Persisted Student: " + student);
```

Ligne 5.5 : `student.setEmail("updated@email.com");`

Detached

- Un objet est dans le state 'detached' quand le 'persistence context' est fermé. Tout changement fait à un objet ne sera propagé à la bd que si l'on utilise la méthode 'merge()'

```
1. tx.begin();
2. student = entityManager.find(Student.class, persistedId);
3. tx.commit();
4. entityManager.close();
5. System.out.println("Persisted Student: " + student);
6. student.setEmail("updated_again@gmail.com");
```

Pour synchroniser les changement à un objet 'detached', il faut utiliser 'merge()'

```
1. //To reattach a detached object
2. student.setEmail("updated_again@gmail.com");
3.
4. entityManager = entityManagerFactory.createEntityManager();
5. tx = entityManager.getTransaction();
6. tx.begin();
7. student = entityManager.merge(student);
8. tx.commit();
9. entityManager.close();
10. System.out.println("Persisted Student: " + student);
```

Removed

- Le statut 'removed' indique que l'instance de l'objet est enlevée de la bd 'delete from ... '

```
1. //Remove an Object from the database
2. entityManager = entityManagerFactory.createEntityManager();
3. tx = entityManager.getTransaction();
4. tx.begin();
5. student = entityManager.find(Student.class, persistedId);
6. entityManager.remove(student);
7. tx.commit();
8. entityManager.close();
```

Entity states

effet de la méthode 'persist()' selon
l'état des entités

- New/Transient => Managed
- Managed/Persistent => Ignorée.
- Detached => Exception
- Removed => Managed

Entity states

effet de la méthode 'merge()' selon
l'état des entités

- New/Transient => Nouvelle instance avec le même état créée
- Managed/Persistent => Ignorée.
- Detached => L'entité existante est chargée de la bd.
L'état est 'fusionné' (merged)
- Removed => IllegalArgumentException

Entity states

effet de la méthode 'remove()' selon
l'état des entités

- New/Transient => Ignorée
- Managed/Persistent => Entité est enlevée.
- Detached => IllegalArgumentException
- Removed => Ignorée

Entity states

effet de la méthode 'refresh()' selon
l'état des entités

- New/Transient => IllegalArgumentException
- Managed/Persistent => Entité est 'reloaded' de la bd.
- Detached => IllegalArgumentException
- Removed => IllegalArgumentException

Synchronisation à la bd

- Commit. Termine le 'unit of work'. Pas de rollback
- Flush: Synchronize sans commit

Fetching

- Il existe 2 façon de 'fetch' les associations. Le 'fetch' consiste à populer variables d'instances d'une entity lorsque celle-ci à des associations à d'autres entités.
- @OneToOne, @OneToMany, @ManyToOne, @ManyToMany
- EAGER: 'Fetch' automatiquement les associations
- LAZY: Ne 'fetch' pas automatiquement les associations.

Fetching

Pourquoi ne voudrais-t'on pas 'fetcher' toutes les associations lorsque l'on hydrate un objet de la base de donnée ?

